פעילות כיתתית – מערכת מעקב אחר חיות בר

Full Stack Wildlife Tracking System

רקע ומטרת הפעילות

רשות הטבע והגנים עוקבת אחרי חיות בר בשמורות הטבע בעזרת קולרי GPS. לכל חיה יש קוד קולר, ונשמרת השמורה האחרונה שבה נצפתה. בנוסף, פקחי הרשות מטפלים בחיות בשמורות השונות, וכל טיפול נרשם במערכת.

בפעילות תבנו בקבוצות מערכת Full Stack שמאפשרת לתחקר את המידע הזה: לקוח Python, שרת Python ומסד נתונים טבלאי.

Python Client
↓ HTTP
Python Server (FastAPI / Flask)
↓
Relational Database (PostgreSQL / MySQL / SQLite)

בכל מקום שבו נדרשת עבודה מול ה-Database אפשר לבחור בין ORM לבין Connector והרצת SQL. פרטי התחברות וכתובות נטענים מקובץ `.env` בלבד, ולא נכתבים ישירות בקוד.

מה נבדק

- קוד תקין ו-Flow מלא בין Client, Server ו-Database.
- **Clean Code:** שמות ברורים, פונקציות קצרות ומבנה קריא.
- **SRP:** אחריות ברורה לכל פונקציה ולכל קובץ.
- **DRY:** בלי שכפול לוגיקה מיותר.
- **עבודת צוות ב-Git:** מאגר משותף, branches, Pull Requests ו-commit-ים מכל חבר צוות.

הדגש הוא על מבנה הקוד ואיכותו, ולא רק על כך שהוא רץ.

נהלי עבודה בקבוצה ו-GitHub

- העבודה מתבצעת בקבוצות של 2–3 סטודנטים.
- **מאגר אחד לכל קבוצה ב-GitHub.** כל חברי הקבוצה מוגדרים בו כ-collaborators.
- **ענף (branch) לכל פיצ'ר או לכל חבר צוות.** המיזוג ל-`main` מתבצע דרך Pull Request בלבד.
- **כל חבר צוות מבצע commit-ים משלו**, עם הודעות שמתארות מה השתנה. כך חלוקת העבודה נראית במאגר.
- **ההגשה היא קישור למאגר** ולא קובץ ZIP.

הנתונים המצורפים

קובץ	תוכן	שימוש
`seed.sql`	יצירת הטבלאות reserves ו-animals וטעינת הנתונים	פיצ'ר 1 – להריץ לפני תחילת העבודה
`reserves.csv`	20 שמורות טבע	פיצ'ר 1 – חלופה ל-seed.sql
`animals.csv`	80 חיות עם קולר	פיצ'ר 1 – חלופה ל-seed.sql
`rangers.csv`	10 פקחים	פיצ'ר 2 – נתונים גולמיים בלי סכמה
`treatments.csv`	150 טיפולים	פיצ'ר 2 – נתונים גולמיים בלי סכמה

את הנתונים של פיצ'ר 1 אפשר לטעון דרך `seed.sql` או דרך קבצי ה-CSV, לפי מה שנוח לכם. הטעינה לא חייבת להיות ב-Python. את הנתונים של פיצ'ר 2 תטענו לטבלאות שאתם עצמכם תתכננו (פרטים בהמשך).

פיצ'ר 1 – Last Seen Reserve (חלק מוכתב)

השאלה שעל המערכת לענות עליה: **באיזו שמורה נצפתה חיה מסוימת לאחרונה?** אפשר לחפש לפי `id` של החיה או לפי `collar_code`.

בחלק הזה כתוב בדיוק אילו קבצים ואילו פונקציות לממש. יש לעמוד בשמות ובאחריות של כל פונקציה.

מבנה הטבלאות הקיימות

טבלת reserves

שדה	משמעות
`id`	Primary Key
`name`	שם השמורה (str)
`region`	אזור (str)
`latitude`	קו רוחב (float)
`longitude`	קו אורך (float)
`description`	תיאור (str)

טבלת animals

שדה	משמעות
`id`	Primary Key
`collar_code`	קוד קולר ייחודי, בפורמט `COL-0042` (str)
`species`	מין החיה (str)
`nickname`	כינוי (str)
`endangered_level`	רמת סכנת הכחדה, 1–5
`last_seen_reserve_id`	FK → reserves.id

מבנה מוכתב – Client

```
client/
├── main.py
├── animal_search.py
├── http_client.py
├── output.py
├── config.py
└── .env
```

בקובץ `.env` מגדירים את כתובת השרת, למשל SERVER_URL. הקובץ `config.py` אחראי לטעון את ההגדרה. אין לכתוב את כתובת השרת ישירות בפונקציות. ודאו שהכתובת והפורט תואמים לשרת שאתם מרימים.

animal_search.py

```
def choose_search_type() -> str:
    ...

def read_animal_id() -> int:
    ...

def read_collar_code() -> str:
    ...
```

- `choose_search_type()` – מציגה Search by ID / Search by collar code, ומחזירה `"id"` או `"collar_code"` לפי בחירת המשתמש. בחירה לא חוקית מבקשת בחירה מחדש.
- `read_animal_id()` – מבקשת ID עד שמתקבל מספר שלם, חיובי ולא ריק.
- `read_collar_code()` – מבקשת קוד קולר עד שמתקבלת מחרוזת לא ריקה שמתחילה ב-`COL-`.

http_client.py

```
def request_last_reserve(search_type: str, search_value) -> dict:
    ...
```

הפונקציה שולחת בקשת GET ל-`animals/last-reserve/`. בחיפוש לפי ID נשלח query parameter בשם `animal_id`, ובחיפוש לפי קוד קולר נשלח query parameter בשם `collar_code`.

output.py

```
def show_last_reserve(result: dict) -> None:
    ...
```

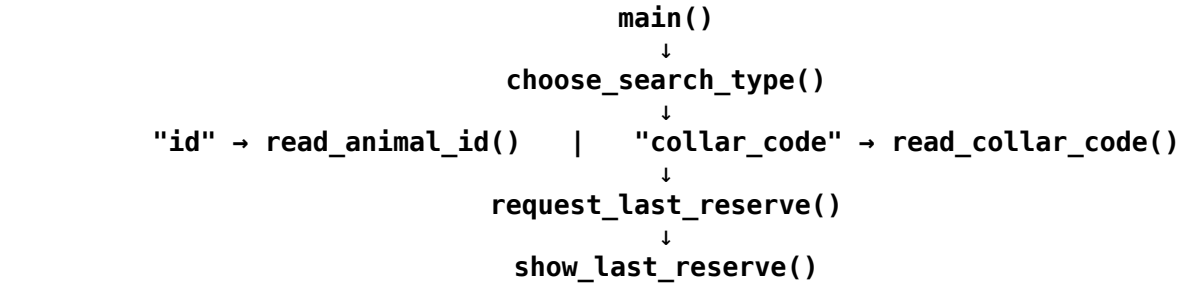
הפונקציה מציגה את תשובת השרת בצורה קריאה, בפורמט הבא. אין להדפיס את ה-JSON כמו שהוא.

```
===== Results =====
Animal ID: <animal_id>
Collar code: <collar_code>
Species: <species> (<nickname>)
Last seen reserve: <reserve_name>
Region: <region>
Description: <description>
!!!!! Done !!!!!
```

main.py

```
def main() -> None:
    ...
```

`main` מנהלת את ה-Flow בלבד. היא לא מבצעת בעצמה ולידציה, בקשות HTTP או הצגה.



Server – מבנה מוכתב

```
server/
├── main.py
├── db.py
├── animal_service.py
├── response_models.py
└── .env
```

אם ה-Framework או דרך העבודה שבחרתם לא דורשים את `response_models.py`, אפשר להשאיר אותו ריק.

db.py

אחראי על יצירת החיבור ל-Database וניהולו, דרך ORM או Connector. פרטי החיבור (host, port, username, password, db name) נשמרים ב-`.env`. את הפונקציות לטעינת המשתנים ולשימוש בהם אתם מגדירים בעצמכם.

animal_service.py

```
def find_last_reserve(search_type: str, search_value):
    ...
```

הפונקציה מחפשת את החיה לפי `id` או לפי `collar_code`, מריצה שאילתת SQL שמחזירה את פרטי החיה ואת פרטי השמורה, ומחזירה את התוצאה.

main.py – Endpoint

ה-Endpoint `GET /animals/last-reserve` מקבל `animal_id` או `collar_code`, קורא ל-`find_last_reserve()` ומחזיר JSON. אם החיה לא נמצאה, יש להחזיר שגיאה מתאימה (404). דוגמה למבנה התשובה:

```
{
  "animal_id": 42,
  "collar_code": "COL-0042",
  "species": "Red Fox",
  "nickname": "Dina",
  "reserve_name": "Mount Meron",
  "region": "North",
  "description": "Highest forested mountain in the Galilee"
}
```

הערכים בדוגמה להמחשה בלבד. כל הנתונים בפועל מגיעים מהקלט או מה-Database.

Flow – פיצ'ר 1

User
↓
Animal ID / collar code
↓
Python Client
↓ HTTP
GET /animals/last-reserve
↓
Python Server → animal_service.py
↓
animals JOIN reserves
↓
JSON Response
↓
Formatted Client Output

פיצ'ר 2 – Ranger Investigation (חלק פתוח)

בחלק הזה אתם מחליטים אילו קבצים, פונקציות וטבלאות להוסיף, תוך שמירה על Clean Code, SRP ו-DRY. מצטרפות שתי ישויות חדשות:

- **פקח (ranger)** – לכל פקח יש לפחות: `id`, `name`, `specialty`, `years_of_service`.
- **טיפול (treatment)** – פקח שטיפל בחיה בשמורה מסוימת. לכל טיפול יש לפחות: `date`, `animal`, `ranger`, `reserve`, `outcome`. הערך של outcome הוא אחד מהשלושה: `healthy`, `treated`, `relocated`.

אפשר להניח שאותו פקח מטפל באותה חיה באותה שמורה פעם אחת בלבד. חשבו איך המבנה שלכם מבטיח את זה.

שאלת התחקיר: בהינתן פקח מסוים (לפי `id` או לפי `name`), כמה חיות ייחודיות הוא טיפל בכל שמורה?

הפיצ'ר מאובטח. רק משתמש רשום עם שם משתמש וסיסמה תקינים יכול לרשום טיפול חדש או להריץ את התחקור.

שלושת ה-Endpoints החדשים

POST /users/register

- מקבל `username` ו-`password` ורושם משתמש חדש.
- המשתמשים נשמרים בקובץ `users.csv` במבנה `username,password_hash`. הסיסמה נשמרת כ-hash בלבד, למשל בעזרת bcrypt.
- שם משתמש שכבר קיים מחזיר שגיאה מתאימה, והמשתמש לא נרשם שוב.

POST /treatments (מאובטח)

- מקבל פרטי טיפול חדש: `date`, `animal_id`, `ranger_id`, `reserve_id`, `outcome`, יחד עם פרטי ההזדהות.

- לפני ההכנסה, השרת בודק שהחיה, הפקח והשמורה קיימים ושה-outcome חוקי. אם לא, מוחזרת שגיאה ושום דבר לא נשמר.
- טיפול כפול (אותו פקח, אותה חיה, אותה שמורה) לא נשמר ומחזיר שגיאה.

GET /rangers/treatments-by-reserve (מאובטח)

- מקבל `ranger_id` או `name` של פקח, יחד עם פרטי ההזדהות.
- מחזיר, לכל שמורה שבה הפקח טיפל, את **שם השמורה** (ולא את ה-id שלה) ואת מספר החיות הייחודיות שטופלו בה.

כלל לשני ה-Endpoints המאובטחים: אם האימות נכשל, אין לגשת ל-Database ואין להחזיר ממנו מידע. האימות מתבצע לפני כל שאילתה.

איך לשלוח את פרטי ההזדהות (headers, body או query params) – זו החלטה שלכם. תעדו אותה ב-`design.txt`.

תכנון פיצ'ר 2 – Database

תכננו את הטבלאות של פקחים וטיפולים, צרו אותן, וטענו אליהן את הקבצים `rangers.csv` ו-`treatments.csv`. המבנה שלכם צריך לאפשר לטעון את הקבצים האלה וגם לענות על שאלת התחקיר.

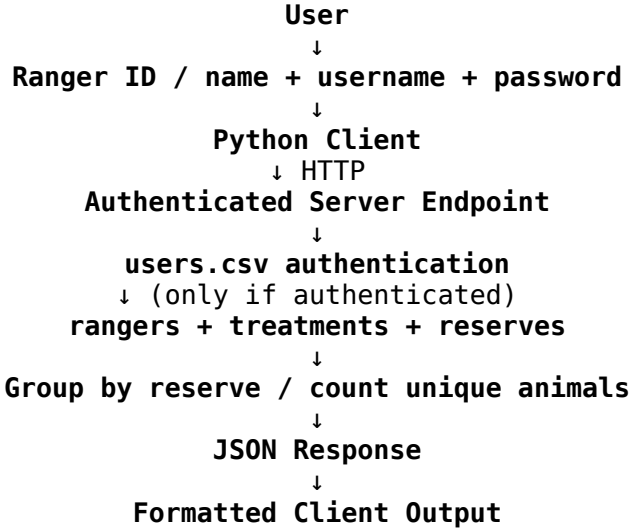
צרו קובץ `sql/design.txt` שמפרט:

1. שמות הטבלאות החדשות.
2. השדות וסוגי הנתונים.
3. Primary Keys.
4. Foreign Keys והטבלאות שאליהן הם מצביעים.
5. איך המבנה מבטיח שאותו טיפול לא נרשם פעמיים.
6. הסבר קצר למה המבנה עונה על שאלת התחקיר, ואיך בחרתם לשלוח את פרטי ההזדהות.

פיצ'ר 2 – Client

- הוסיפו לתפריט הראשי אפשרויות לרישום משתמש, לרישום טיפול חדש ולתחקור פקח.
- רישום משתמש: קבלת username ו-password, עם ולידציה שהם לא ריקים.
- רישום טיפול: קבלת כל השדות עם ולידציה – מזהים הם מספרים שלמים חיוביים, התאריך בפורמט `YYYY-MM-DD`, ו-outcome הוא אחד משלושת הערכים החוקיים.
- תחקור פקח: חיפוש לפי `id` או `name`.
- לפני כל פעולה מאובטחת: קבלת username ו-password וולידציה שהם לא ריקים.
- הצגת כל התוצאות בצורה קריאה, לא כהדפסה ישירה של JSON. שגיאות מהשרת מוצגות כהודעה ברורה למשתמש.

פיצ'ר 2 – Flow



סיכום ה-Endpoints

Endpoint	Method	מאובטח	קלט	פלט
`animals/last-reserve/`	GET	לא	animal_id או collar_code	פרטי החיה והשמורה האחרונה
`users/register/`	POST	לא	username, password	אישור או שגיאה
`treatments/`	POST	כן	פרטי הטיפול + הזדהות	אישור או שגיאה
`rangers/treatments-by-/reserve`	GET	כן	ranger_id או name + הזדהות	שם שמורה ומספר חיות ייחודיות

בסיום הפעילות

- **פיצ'ר 1:** חיפוש חיה לפי id או collar_code והצגת השמורה האחרונה שבה נצפתה.
- **פיצ'ר 2:** רישום משתמש, רישום טיפול מאובטח, ותחקור פקח מאובטח שמציג את מספר החיות הייחודיות בכל שמורה.
- ה-Client וה-Server תואמים ב-Endpoint, ב-HTTP Method ובמבנה ה-Request וה-Response.
- המערכת נבדקת כיחידה אחת: Client → Server → Database → Server → Client.

הגשה

1. ודאו שכל העבודה ממוזגת ל-`main` דרך Pull Requests.
2. המאגר מכיל את התיקיות `client`, `server` ו-`sql`.
3. הגישו את הקישור למאגר, יחד עם שמות כל חברי הקבוצה.